



Formal Verification Report



Aave V4 Spoke

March 2026

Prepared for Aave Labs

Table of contents

Project Summary	3
Project Scope	3
Project Overview	3
Assessment Methodology	4
Formal Verification	4
Security Context	4
Collaboration and Fix Validation	5
Findings Summary	6
Severity Matrix	6
Detailed Findings	7
Medium Severity Issues	8
M-01 Changing collateral factor from non-zero to 0 can cause liquidations to revert	8
Low Severity Issues	9
L-01 Increase of debt position without health check due to unsafe casting	9
Formal Verification	10
Verification Notations	10
General Assumptions and Simplifications	10
Properties Overview	11
Detailed Properties	12
Spoke	12
P-01. Accounting & Integrity Functional level	12
P-02. Only non-zero changes	13
P-03. Spoke Hub consistency	13
P-04. Health check validity	14
P-05. Integrity of fields	15
P-06. Liquidation Integrity	16
P-07. Liquidation Report Deficit integrity	17
Disclaimer	18
About Certora	18

Project Summary

Project Scope

Project Name	Repository (link)	Platform
Aave V4	https://github.com/aave/aave-v4	EVM

Project Overview

This document describes the specification and verification of **Aave V4 Spoke** using the Certora Prover. The document is part of three documents describing the work undertaken from **July 28, 2025** to **March 09, 2026**.

This is one of three formal verification reports, focusing on the **Spoke** component. The primary verification target is `Spoke.sol`. The following internal libraries are exercised as part of the Spoke verification:

```
src/spoke/Spoke.sol
src/spoke/libraries/UserPositionDebt.sol
src/spoke/libraries/LiquidationLogic.sol
src/spoke/libraries/PositionStatusMap.sol
src/spoke/libraries/ReserveFlagsMap.sol
src/spoke/libraries/KeyValueList.sol
```

Standalone property verification of the library contracts is documented in the companion Libraries Formal Verification Report. The Hub is represented as a symbolic abstraction based on properties proven in the companion Hub Formal Verification Report.

The Certora Prover demonstrated that the implementation of the Solidity contracts of the Spoke is correct with respect to the formal rules written by the Certora team. During the verification process, the Certora team discovered bugs in the Solidity contracts code, as listed on the following pages.

Assessment Methodology

Formal Verification

We performed verification of the **AAVE V4** protocol using the Certora verification tool which is based on Satisfiability Modulo Theories (SMT). In short, the Certora verification tool works by compiling formal specifications written in the [Certora Verification Language \(CVL\)](#) and **AAVE's** implementation source code written in Solidity.

More information about Certora's tooling can be found in the [Certora Technology Whitepaper](#).

If a property is verified with this methodology it means the specification in CVL holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules. A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants. Inductive invariants are proved by induction on the structure of a smart contract. We use constructors as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.

Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective constructor. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant.

Security Context

The properties verified in this report target the subset of identified threats that can be addressed at the Spoke contract level:

- accounting and position integrity, ensuring each operation touches only the intended user's state and only by non-zero amounts ([P-01](#), [P-02](#));

- consistency between the Spoke's per-user accounting and the aggregate state held by the Hub ([P-03](#));
- overcollateralization and health-factor enforcement, ensuring the health check is applied wherever an operation could reduce it and that positions below threshold can only recover ([P-04](#));
- integrity of position and reserve fields, including the invariant that debt cannot exist without collateral ([P-05](#)); and
- liquidation safety and correctness, including that healthy accounts cannot be liquidated, that calculated amounts match realized state changes, and that deficit reporting is sound ([P-06](#), [P-07](#)).

Access control enforcement, such as only an authorized position manager or a caller verified via `checkCanCall` may mutate a user's position, is covered within these property groups (P-01). Hub-level properties are verified in the companion Hub Formal Verification Report, and library-level properties are verified in the companion Libraries Formal Verification Report. System-level concerns such as oracle feed quality, economic attack surfaces, blockchain infrastructure risks, and admin compromise are outside the scope of this report.

Collaboration and Fix Validation

Collaboration with the development team runs throughout the engagement to confirm expected behaviors, clarify design assumptions, and ensure accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow-up review validates applied fixes and verifies that no regressions or secondary issues have been introduced.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	1	1	1
Low	1	1	1
Informational	-	-	-
Total	2	2	2

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
M-01	Changing collateral factor from non-zero to 0 can cause liquidations to revert	Medium	Fixed
L-01	Increase of debt position without health check due to unsafe casting	Low	Fixed

Medium Severity Issues

M-01 Changing collateral factor from non-zero to 0 can cause liquidations to revert

Severity: Medium	Impact: High	Likelihood: Low
Files: Spoke.sol	Status: Fixed	Violated Property: noCollateralNoDebt

Description: CollateralFactor of an asset can be changed from non-zero value to 0.

This is allowed in `addDynamicReserveConfig()` and `updateDynamicReserveConfig()` so after the admin makes this change users can update their config via `updateUserDynamicConfig()`.

Exploit Scenario: In case users have borrowed debt using collateral with collateral factor being 0 they no longer can be liquidated since the liquidation call validates that the liquidated collateral is with a non-zero collateral factor.

Recommendations: We recommend never allowing asset collateral factors to be changed from non-zero to 0.

Customer's response: Fixed in `v0.5.1`.

Fix Review: Fix confirmed on [PR#802](#).

Low Severity Issues

L-01 Increase of debt position without health check due to unsafe casting

Severity: Low	Impact: High	Likelihood: Low
Files: UserPositionDebt.sol	Status: Fixed	Violated Property: increaseCollateralOrReduceDebtFunctions

Description: The computation `newPremiumShares * drawnIndex` is casted to int without checks. It is unlikely that this multiplication will reach `max_uint255` but if it does, then the offset will become a negative number representing debt

```
JavaScript
function getPremiumDelta(
  ...
) internal view returns (IHubBase.PremiumDelta memory) {
  ...
  int256 newPremiumOffsetRay = (newPremiumShares * drawnIndex).signedSub(
    premiumDebtRay - restoredPremiumRay
  );
}
```

Exploit Scenario: Triggering this issue would require a very rare, but technically possible, precondition, that `newPremiumShares * drawnIndex` reaches a value that is greater than the maximum int256, but smaller than the maximum uint256

Recommendations: Use safe cast.

Customer's response:

Fixed in [PR#1071](#).

Fix Review: Fix confirmed.

Formal Verification

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue

General Assumptions and Simplifications

- We use our own implementation of `Math.mulDiv`, `Math.mulDivUp` and `Math.mulDivDown`. These are equivalent to the originals and better suited for verification purposes.
- Loops are inherently difficult for formal verification. We handle loops by unrolling them a specific amount of times. We use a **loop_iter of 3**, unrolling each loop up to three times.
- Underlying assets are assumed to be standard ERC20, specifically:
 - a. No fees on transfer
 - b. Without double entries
 - c. Decimals between 6 and 18
- The price of assets is assumed to be non-zero.
- The verified Solidity contracts are compiled with solc8.28 (with via-ir).

Properties Overview

ID	Contract	Title	Impact
P-01	Spoke	Accounting & Integrity Functional level	Direct fund loss, broken economic model, potential insolvency
P-02	Spoke	Only non-zero changes	Direct fund loss, broken economic model, potential insolvency
P-03	Spoke	Spoke Hub consistency	Direct fund loss, broken economic model, potential insolvency
P-04	Spoke	Health check validity	Direct fund loss, broken economic model, potential insolvency
P-05	Spoke	Integrity of fields	Direct fund loss, broken economic model, potential insolvency
P-06	Spoke	Liquidation Integrity	Direct fund loss, potential insolvency
P-07	Spoke	Liquidation Report Deficit	Denial of service, potential insolvency

Detailed Properties

Spoke

Assumptions and modularity:

- We prove properties P01, P02, P04–P06 with a symbolic representation of Hub that is based on properties proven on Hub.
- Function multicall() , extSload(), extSloads() are out of scope.
- Function liquidationCall() is out of scope for property groups P–01 through P–05 and is verified separately in P–06.
- In P–04 we assume up to three debt reserves and three collateral reserves.

P–01. Accounting & Integrity Functional level

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
supply_noChangeToOther	Formally Verified	Supply integrity - only suppliedShares changes for the user	Run link
withdraw_noChangeToOther	Formally Verified	Withdraw integrity - only suppliedShares changes for the user	
borrow_noChangeToOther	Formally Verified	Borrow integrity - drawnShares increases, premiumShares may change	
repay_noChangeToOther	Formally Verified	Repay integrity - drawnShares decreases, suppliedShares unchanged	
onlyPositionManagerCanChange	Formally Verified	Only position manager can change the user's position, or that the caller was verified via the checkCanCall function	
userIntegrity	Formally Verified	Only one user's account is updated and used in a single operation (beside liquidationCall and multicall)	Run link

paused_noChange	Formally Verified	If paused - no change	Run link
paused_noLiquidation	Formally Verified	When paused (collateral or debt) then no liquidation	
frozen_onlyReduceDebtAndCollateral	Formally Verified	If frozen - functions can only reduce debt and reduce collateral	
deterministicUserDebtValue	Formally Verified	User debt value is deterministic	Run link

P-02. Only non-zero changes

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
nothingForZero_supply	Formally Verified	Supply operation increases user's supplied shares and transfers tokens from user	Run link
nothingForZero_withdraw	Formally Verified	Withdraw operation decreases user's supplied shares and transfers tokens to user	
nothingForZero_borrow	Formally Verified	Borrow operation increases user's drawn shares and transfers tokens to user	
nothingForZero_repay	Formally Verified	Repay operation decreases user's drawn shares and transfers tokens from user	

P-03. Spoke Hub consistency

Status: Formally Verified

Rule Name	Status	Description	Link to rule
-----------	--------	-------------	--------------

report

userDrawnShareConsistency	Formally Verified	The sum of users' drawn shares for a reserveld are consistent with the Hub drawn shares for the Spoke	Run link
userPremiumShareConsistency	Formally Verified	The sum of users' premium shares for a reserveld are consistent with the Hub premium shares for the Spoke	
userPremiumOffsetConsistency	Formally Verified	The sum of users' premium offset for a reserveld are consistent with the Hub premium offset for the Spoke	
userSuppliedShareConsistency	Formally Verified	The sum of users' supplied shares for a reserveld are consistent with the Hub supplied shares for the Spoke	
repay_debtDecrease	Formally Verified	Verify that the repay function reduces the debt of a user	Run link
repay_zeroDebt	Formally Verified	Verify that if the user has no drawn shares, then there are no premium shares or offset.	

P-04. Health check validity

Status: Formally Verified After Fix

Rule Name	Status	Description	Link to rule report
increaseCollateralOrReduceDebtFunctions	Formally Verified After Fix	Verify functions that increase collateral or reduce debt. These functions do not need to perform a health check. This property was violated due to L-01	Run link
userHealthChecked	Formally Verified	Verify that the health factor is checked after updates to the user position which can reduce the	Run link

		health factor	
belowThresholdReverting	Formally Verified	Functions: <code>updateUserDynamicConfig()</code> and <code>borrow()</code> revert when health factor is below threshold	Run link
userHealthBelowThresholdCanOnlyIncreaseHealthFactor	Formally Verified	Verify that the health factor can only increase if the health factor is below the threshold	Run link setUsingAsCollateral updateUserRiskPremium
userHealthAboveThreshold	Formally Verified	Verify that the health factor is above the threshold after any operation. Verified on all functions but <code>setUsingAsCollateral</code> and <code>borrow</code>	Run link updateUserRiskPremium updateUserDynamicConfig withdraw repay

P-05. Integrity of fields

Status: Formally Verified After Fix

Rule Name	Status	Description	Link to rule report
noCollateralNoDebt	Formally Verified After Fix	If there is no collateral, then there is no debt. This property was violated due to M-01	Run link supply
drawnSharesZero	Formally Verified	If there are no drawn shares, then there are no premium shares or offset	Run link

updateUserRiskPremium_preservesPremiumDebt	Formally Verified	updateUserRiskPremium preserves premium debt	Run link
collateralFactorNotZero	Formally Verified	Collateral factor is not zero once set to a non-zero value	
isBorrowingIFFdrawnShares	Formally Verified	Borrowing flag is set if and only if there are drawn shares	
validReserveld	Formally Verified	Valid reserve state for all users and reserves	
hubAssetIdToReserveldIntegrity	Formally Verified	Hub asset ID to reserve ID integrity	
uniqueAssetIdPerReserveld	Formally Verified	The assetId and hub are unique to a reserveld	
realizedPremiumRayConsistency	Formally Verified	The realized premium ray is consistent with the premium shares and drawn index	
drawnSharesRiskEQPremiumShares	Formally Verified	The drawn shares are equal to the premium shares multiplied by the risk premium	
dynamicConfigKeyConsistency	Formally Verified	The dynamic config key is consistent with the reserve dynamic config key	

P-06. Liquidation Integrity

Status: Formally Verified After Fix

Rule Name	Status	Description	Link to rule report
onlyOneUserDebtChanges_liquidationCall	Formally Verified	Only one user's debt changes during liquidation	Run link
liquidationIntegrity	Formally Verified	Liquidation integrity - calculated amounts match actual state changes	Run link

borrowingFlagSetIFFdrawnShares_liquidationCall	Formally Verified	Borrowing flag set if and only if drawn shares exist	Run link
healthyAccountCannotBeLiquidated	Formally Verified	Healthy account cannot be liquidated	
noChangeToOtherAccounts_liquidationCall	Formally Verified	No change to other accounts during liquidation. In the liquidated account debt can be decreased to zero on report deficit, however other collateral cannot change at all	
monotonicityOfDebtDecrease_collateralIncrease	Formally Verified After Fix	Monotonicity of debt decrease during liquidation: If collateral increases for the liquidator, then debt must decrease for the liquidated user. This property was violated due to C-01 in the companion Libraries Formal Verification Report	Run link

P-07. Liquidation Report Deficit integrity

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
moreThanOneCollateral_noReportDeficit	Formally Verified	More than one collateral - no report deficit	Run link
moreCollateralThanDebt_noReportDeficit	Formally Verified	More collateral value than debt value - no report deficit	

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.